

STOREFILLER — Technical Architecture & Implementation Guide

B2B Wholesale Restocking Marketplace · Zero-Budget Modular Monolith · Mobile-Only

0. Scope & Assumptions

- **Team:** 4 developers — 3 Backend (BE-1, BE-2, BE-3) + 1 Mobile (MOB-1, React Native/Expo). No web frontend devs needed since the brief specifies mobile-only for Buyers, Agents, and Admin.
- **Model:** Unlike a multi-vendor marketplace, Storefiller has a **single central catalog** (one warehouse/admin managing products), matching the "Instamart for store owners" framing in your screenshots. This removes an entire seller/farmer layer of complexity compared to Kisaan Connect — fewer tables, simpler FSM, faster 4-week build.
- **Roles:** BUYER (store owner), AGENT (delivery), ADMIN (warehouse ops) — all inside one Expo app, routed by role.
- **Delivery model:** Since buyers are restocking *their own shop*, there is one fixed delivery address per buyer (the shop itself) — captured once at onboarding, not re-entered per order. This is what your onboarding flow (shop details → address) already implies.

1. System Architecture Diagram

flowchart TB

```
subgraph Mobile["📱 React Native (Expo) — Single App, Role-Routed"]
  Buyer["Buyer Screens\n(Onboarding, Catalog, Cart, Orders)"]
  Agent["Agent Screens\n(Deliveries, Live Location, Geofence Confirm)"]
  AdminM["Admin Screens\n(Catalog Mgmt, Order Ops, Settlements)"]
end

subgraph API["⚙️ Node.js + Express — Modular Monolith (Railway)"]
  AuthMod["Auth Module\n(Firebase verify + JWT)"]
  CatalogMod["Catalog + pg_trgm Search"]
  CartMod["Cart Module"]
  OrderMod["Order FSM + Inventory Lock"]
  PayMod["Payments + Wallet"]
  DeliveryMod["Delivery + Tracking + Geofence"]
end
```

```
Cron["node-cron\n(single instance)"]
end
```

```
subgraph DB["🐘 Neon Serverless PostgreSQL"]
  PG["pg_trgm + PostGIS\n(single branch)"]
end
```

```
subgraph Ext["🍷 Free-Tier External Services"]
  FB["Firebase Auth Admin SDK\n(Phone OTP)"]
  Cloud["Cloudinary\n(direct mobile upload)"]
  Razor["Razorpay\n(Standard, per-txn)"]
  Mail["Email OTP fallback\n(Resend free tier)"]
end
```

```
Buyer -- "REST + JWT" --> API
Agent -- "REST + JWT, POST loc every 15s" --> API
AdminM -- "REST + JWT" --> API
Buyer -- "signed direct upload" --> Cloud
Cloud -- "upload webhook" --> API
Agent -- "poll GET /tracking every 15s" --> API
```

```
AuthMod <--> FB
AuthMod --> Mail
PayMod <--> Razor
API <--> PG
Cron --> PG
```

Key architectural decisions baked into this diagram:

- One Express process, one Railway service, one Neon branch — no microservices, no message queue. At B2B scale (tens to low hundreds of shop-owners, not consumer volume) this is the right call and keeps the whole stack at ₹0/month.

- Cloudinary uploads go **mobile** → **Cloudinary directly** (signed unsigned-preset upload), never through your API — this is the single biggest bandwidth saver on Railway's free tier.
 - Tracking is **pull, not push**: the agent app POSTs location, the buyer app polls a lightweight GET /tracking endpoint. No WebSocket server, no Redis pub/sub.
-

2. Database Schema

2.1 Prisma Schema (schema.prisma)

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

enum Role {
  BUYER
  AGENT
  ADMIN
}

enum ProductUnit {
  KG
  GRAM
  LITRE
  ML
  PACK
  PIECE
```

BOX

}

enum PaymentMethod {

WALLET

UPI

CARD

COD

}

enum PaymentStatus {

NOT_APPLICABLE

PENDING

CAPTURED

FAILED

REFUNDED

}

enum OrderStatus {

PENDING_PAYMENT

CONFIRMED

DISPATCHED

OUT_FOR_DELIVERY

DELIVERED

CANCELLED

}

enum WalletTxnType {

CREDIT

DEBIT

```
}
```

```
enum TicketStatus {  
    OPEN  
    IN_PROGRESS  
    RESOLVED  
}
```

```
enum JobStatus {  
    PENDING  
    PROCESSED  
    FAILED  
}
```

```
model User {  
    id          String  @id @default(uuid())  
    phone       String  @unique  
    email       String? @unique  
    role        Role  
    firebaseUid String? @unique  
    isActive    Boolean @default(true)  
    pushEnabled Boolean @default(true)  
    orderUpdatesOptIn Boolean @default(true)  
    promoOptIn  Boolean @default(false)  
    createdAt   DateTime @default(now())  
    updatedAt   DateTime @updatedAt  
  
    shopProfile ShopProfile?  
    agentProfile AgentProfile?  
    deviceTokens DeviceToken[]
```

```

cartItems    CartItem[]
orders       Order[]
wallet       BuyerWallet?
ratings      OrderRating[]
tickets      SupportTicket[]
referralCode ReferralCode?
referralsMade Referral[]    @relation("Referrer")
referralsRecv Referral[]    @relation("Referred")
auditLogs    AuditLog[]

```

```

@@index([phone])

```

```

}

```

```

model ShopProfile {
  id          String @id @default(uuid())
  userId      String @unique
  user        User   @relation(fields: [userId], references: [id])
  shopName    String
  ownerName   String
  shopPhotoUrl String?
  addressLine String
  city        String
  state       String
  pincode     String
  latitude    Float
  longitude   Float
  // `location geography(Point,4326)` added via raw SQL migration below —
  // kept in sync with latitude/longitude on every write from the app layer.
  location    Unsupported("geography(Point,4326)")?
  createdAt   DateTime @default(now())

```

```
    updatedAt DateTime @updatedAt
}
```

```
model AgentProfile {
  id          String  @id @default(uuid())
  userId      String  @unique
  user        User    @relation(fields: [userId], references: [id])
  vehicleType String?
  isAvailable Boolean @default(true)
  lastLatitude Float?
  lastLongitude Float?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

```
model DeviceToken {
  id      String  @id @default(uuid())
  userId  String
  user    User    @relation(fields: [userId], references: [id])
  token   String  @unique
  platform String
  isActive Boolean @default(true)
  createdAt DateTime @default(now())

  @@index([userId])
}
```

```
model Category {
  id      String  @id @default(uuid())
  name    String
```

```

slug      String  @unique
iconUrl   String?
parentId  String?
parent    Category? @relation("CategoryTree", fields: [parentId], references: [id])
subcategories Category[] @relation("CategoryTree")
displayOrder Int    @default(0)
isActive  Boolean   @default(true)
products  Product[]
}

```

```

model Product {
  id      String  @id @default(uuid())
  name    String
  brand   String?
  sku     String  @unique
  categoryId String
  category Category @relation(fields: [categoryId], references: [id])
  unit    ProductUnit
  unitSize String
  caseQty Int      @default(1)
  mrp     Decimal  @db.Decimal(10, 2)
  sellingPrice Decimal @db.Decimal(10, 2)
  imageUrl String?
  description String?
  isActive Boolean  @default(true)
  version  Int       @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  inventory ProductInventory?
}

```



```
cartItems CartItem[]
orderItems OrderItem[]
```

```
@@index([categoryId])
}
```

```
model ProductInventory {
  productId    String  @id
  product      Product @relation(fields: [productId], references: [id])
  quantityAvailable Int   @default(0)
  quantityReserved Int   @default(0)
  lowStockThreshold Int   @default(10)
  updatedAt    DateTime @updatedAt
}
```

```
model CartItem {
  id      String  @id @default(uuid())
  buyerId String
  buyer   User    @relation(fields: [buyerId], references: [id])
  productId String
  product  Product @relation(fields: [productId], references: [id])
  quantity Int
  addedAt  DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@unique([buyerId, productId])
}
```

```
model Order {
  id      String  @id @default(uuid())
```

```

buyerId      String
buyer        User      @relation(fields: [buyerId], references: [id])
agentId      String?
status       OrderStatus @default(PENDING_PAYMENT)
paymentMethod PaymentMethod
paymentStatus PaymentStatus @default(NOT_APPLICABLE)
subtotalAmount Decimal   @db.Decimal(10, 2)
deliveryFee   Decimal   @default(0) @db.Decimal(10, 2)
discountAmount Decimal   @default(0) @db.Decimal(10, 2)
totalAmount   Decimal   @db.Decimal(10, 2)
deliveryAddress Json      // snapshot of ShopProfile address at order time
version       Int        @default(1)
placedAt      DateTime   @default(now())
confirmedAt   DateTime?
dispatchedAt  DateTime?
deliveredAt   DateTime?
cancelledAt   DateTime?
cancellationReason String?

```

```

items      OrderItem[]
payment     Payment?
rating      OrderRating?
tracking    LiveTracking?
locationLogs DeliveryLocationLog[]
settlement  Settlement?

```

```

@@index([buyerId])
@@index([status])
@@index([agentId])

```

```

}

```

```

model OrderItem {
  id          String @id @default(uuid())
  orderId     String
  order       Order  @relation(fields: [orderId], references: [id])
  productId   String
  product     Product @relation(fields: [productId], references: [id])
  productNameSnap String
  unitPriceSnap Decimal @db.Decimal(10, 2)
  quantity    Int
  lineTotal   Decimal @db.Decimal(10, 2)

  @@index([orderId])
}

```

```

model Payment {
  id          String @id @default(uuid())
  orderId     String @unique
  order       Order  @relation(fields: [orderId], references: [id])
  razorpayOrderId String?
  razorpayPaymentId String? @unique
  razorpaySignature String?
  amount      Decimal @db.Decimal(10, 2)
  status      PaymentStatus @default(PENDING)
  capturedAt  DateTime?
  createdAt   DateTime @default(now())
}

```

```

model BuyerWallet {
  buyerId String @id
}

```

```

buyer    User    @relation(fields: [buyerId], references: [id])
balance  Decimal @default(0) @db.Decimal(10, 2)
updatedAt DateTime @updatedAt

```

```

transactions WalletTransaction[]
}

```

```

model WalletTransaction {
  id          String      @id @default(uuid())
  buyerId     String
  wallet       BuyerWallet @relation(fields: [buyerId], references: [buyerId])
  type         WalletTxnType
  amount       Decimal     @db.Decimal(10, 2)
  balanceAfter Decimal     @db.Decimal(10, 2)
  referenceOrderId String?
  createdAt    DateTime    @default(now())

  @@index([buyerId])
}

```

// Single row per order — cheap O(1) reads for the 15s polling endpoint.

```

model LiveTracking {
  orderId String @id
  order   Order  @relation(fields: [orderId], references: [id])
  agentId String
  latitude Float
  longitude Float
  location Unsupported("geography(Point,4326)")?
  updatedAt DateTime @updatedAt
}

```

// Append-only history, pruned by cron every 30 days — used for audit/replay, not for live reads.

```
model DeliveryLocationLog {
  id      String  @id @default(uuid())
  orderId String
  order   Order   @relation(fields: [orderId], references: [id])
  agentId String
  latitude Float
  longitude Float
  recordedAt DateTime @default(now())

  @@index([orderId])
}
```

```
model Settlement {
  id      String  @id @default(uuid())
  orderId String  @unique
  order   Order   @relation(fields: [orderId], references: [id])
  agentId String
  amount  Decimal @db.Decimal(10, 2)
  status  String  @default("PENDING") // PENDING | PAID
  paidAt  DateTime?
  createdAt DateTime @default(now())

  @@index([agentId])
}
```

```
model OrderRating {
  orderId String @id
  order   Order  @relation(fields: [orderId], references: [id])
}
```

```

buyerId String
buyer User @relation(fields: [buyerId], references: [id])
rating Int
comment String?
createdAt DateTime @default(now())
}

```

```

model SupportTicket {
  id String @id @default(uuid())
  buyerId String
  buyer User @relation(fields: [buyerId], references: [id])
  subject String
  message String
  status TicketStatus @default(OPEN)
  createdAt DateTime @default(now())
  resolvedAt DateTime?

  @@index([buyerId])
}

```

```

model ReferralCode {
  userId String @id
  user User @relation(fields: [userId], references: [id])
  code String @unique
  createdAt DateTime @default(now())
}

```

```

model Referral {
  id String @id @default(uuid())
  referrerId String
}

```

```

referrer    User    @relation("Referrer", fields: [referrerId], references: [id])
referredId  String  @unique
referred    User    @relation("Referred", fields: [referredId], references: [id])
rewardCredited Boolean @default(false)
createdAt   DateTime @default(now())
}

```

```

// Restart-safety for in-memory node-cron: anything that must survive a
// server restart gets written here BEFORE the triggering action, and
// replayed on boot. Same pattern as reconciliation crons elsewhere.

```

```

model JobRecovery {
  id      String  @id @default(uuid())
  jobType  String
  referenceId String
  status   JobStatus @default(PENDING)
  payload  Json
  createdAt DateTime @default(now())
  processedAt DateTime?

```

```

  @@index([status])
}

```

```

model AuditLog {
  id      String  @id @default(uuid())
  adminId  String
  admin    User    @relation(fields: [adminId], references: [id])
  action   String
  entityType String
  entityId  String
  metadata  Json?

```

```
createdAt DateTime @default(now())
```

```
@@index([entityType, entityId])
```

```
}
```

2.2 Raw SQL Migration Supplement

Prisma can't express pg_trgm/PostGIS operator classes or GIST indexes natively, so these run as a supplementary migration (prisma migrate dev --create-only + hand-edit):

```
-- Extensions (run once, Day 1)
```

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
-- Fuzzy product search
```

```
CREATE INDEX products_name_trgm ON "Product" USING GIN (name gin_trgm_ops);
```

```
CREATE INDEX products_brand_trgm ON "Product" USING GIN (brand gin_trgm_ops);
```

```
-- Geospatial columns (declared as Unsupported() in Prisma, added here)
```

```
ALTER TABLE "ShopProfile" ADD COLUMN IF NOT EXISTS location  
geography(Point,4326);
```

```
ALTER TABLE "LiveTracking" ADD COLUMN IF NOT EXISTS location  
geography(Point,4326);
```

```
CREATE INDEX shop_profile_location_gist ON "ShopProfile" USING GIST (location);
```

```
-- Verify both extensions are active before any dependent module starts
```

```
SELECT extname FROM pg_extension WHERE extname IN ('pg_trgm','postgis');
```

App-layer writes latitude/longitude (plain floats, easy to read in Prisma Client) **and** location (geography, via one raw \$executeRaw call in the same transaction) on every insert/update — this avoids needing a DB trigger while still getting accurate ST_Distance for the geofence check.

3. API Contracts

All routes prefixed /api/v1. Auth column: Public (no token), Buyer/Agent/Admin (role-scoped JWT).

Auth

Method	Path	Auth	Request	Response
POST	/auth/token/verify	Public	{ firebaseIdToken }	{ accessToken, refreshToken, user }
POST	/auth/email-otp/send	Public	{ email }	{ message }
POST	/auth/email-otp/verify	Public	{ email, otp }	{ accessToken, refreshToken, user }
POST	/auth/refresh	Public (refresh token)	{ refreshToken }	{ accessToken, refreshToken }
POST	/auth/logout	Buyer/Agent/Admin	{ }	{ message }

Firebase handles the OTP UI and SMS delivery client-side; the app never talks OTP directly to your backend. The mobile SDK returns a Firebase ID token, which your backend verifies once via firebase-admin's `verifyIdToken()`, then issues its **own** RS256 JWT for every subsequent call — this decouples all later auth from Firebase and lets the email-OTP fallback plug into the exact same session layer.

Shop Onboarding

Method	Path	Auth	Request	Response
POST	/shop/profile	Buyer	{ shopName, ownerName, addressLine, city, state, pincode, latitude, longitude }	{ shopProfile }
PATCH	/shop/profile	Buyer	partial fields	{ shopProfile }
POST	/shop/photo/upload-signature	Buyer	{ }	{ timestamp, signature, apiKey, cloudName, folder }
PATCH	/shop/photo/confirm	Buyer	{ cloudinaryUrl, publicId }	{ shopProfile }

Catalog & Search

Method	Path	Auth	Request	Response
GET	/categories	Buyer	—	{ categories[] } (tree, subcategories nested)

Method	Path	Auth	Request	Response
GET	/products	Buyer	query: q, categoryId, page, limit	{ data[], total, page }
GET	/products/:id	Buyer	—	{ product, inventory }
POST	/admin/products	Admin	{ name, brand, sku, categoryId, unit, unitSize, caseQty, mrp, sellingPrice, imageUrl, description }	{ product }
PATCH	/admin/products/:id	Admin	partial + version	{ product } (409 on stale version)
PATCH	/admin/products/:id/inventory	Admin	{ quantityAvailable, version }	{ inventory }
POST	/admin/categories	Admin	{ name, slug, parentId?, imageUrl? }	{ category }

Cart

Method	Path	Auth	Request	Response
GET	/cart	Buyer	—	{ items[], subtotal }
POST	/cart/items	Buyer	{ productId, quantity }	{ items[] }
PATCH	/cart/items/:productId	Buyer	{ quantity }	{ items[] }
DELETE	/cart/items/:productId	Buyer	—	{ items[] }
DELETE	/cart	Buyer	—	{ message }

Orders

Method	Path	Auth	Request	Response
POST	/orders	Buyer	{ paymentMethod }	{ order } — built server-side from the live cart , never from client-supplied line items/prices
GET	/orders	Buyer	query: status, page	{ data[], total }
GET	/orders/:id	Buyer/Agent/Admin	—	{ order, items, tracking }

Method	Path	Auth	Request	Response
PATCH	/orders/:id/cancel	Buyer	{ reason }	{ order }
GET	/orders/:id/tracking	Buyer	—	{ latitude, longitude, status, updatedAt }
GET	/orders/:id/receipt	Buyer	—	PDF stream (generated on-demand, not stored)
POST	/orders/:id/rating	Buyer	{ rating, comment }	{ rating }

Admin — Order Ops

Method	Path	Auth	Request	Response
GET	/admin/orders	Admin	query: status	{ data[] }
PATCH	/admin/orders/:id/confirm	Admin	{ }	{ order } — used for COD manual review
PATCH	/admin/orders/:id/assign-agent	Admin	{ agentId }	{ order }

Agent

Method	Path	Auth	Request	Response
GET	/agent/orders	Agent	—	assigned/available orders
PATCH	/agent/orders/:id/pickup	Agent	{ }	{ order } → OUT_FOR_DELIVERY
POST	/agent/orders/:id/location	Agent	{ latitude, longitude, accuracy }	{ ok: true } — called every 15s
PATCH	/agent/orders/:id/deliver	Agent	{ latitude, longitude }	{ order } — 422 if outside 200m geofence

Payments & Wallet

Method	Path	Auth	Request	Response
POST	/payments/razorpay/order	Buyer	{ orderId }	{ razorpayOrderId, amount, currency, keyId }

Method	Path	Auth	Request	Response
POST	/payments/razorpay/webhook	Public (HMAC verified)	Razorpay payload	200
GET	/wallet	Buyer	—	{ balance }
GET	/wallet/transactions	Buyer	—	{ data[] }
POST	/wallet/topup/razorpay-order	Buyer	{ amount }	{ razorpayOrderId }

Notifications, Support, Referral

Method	Path	Auth	Request	Response
PATCH	/users/me/notification-preferences	Buyer/Agent	{ pushEnabled, orderUpdatesOptIn, promoOptIn }	{ user }
POST	/users/me/device-token	Buyer/Agent/Admin	{ token, platform }	{ message }
POST	/support/tickets	Buyer	{ subject, message }	{ ticket }
GET	/support/tickets	Buyer	—	{ data[] }
GET	/referral/code	Buyer	—	{ code }
GET	/referral/history	Buyer	—	{ data[] }

Admin — Ops

Method	Path	Auth	Request	Response
GET	/admin/dashboard	Admin	—	{ pendingOrders, pendingSettlements, lowStock[] }
GET	/admin/settlements	Admin	—	{ data[] }
PATCH	/admin/settlements/:id/pay	Admin	{ payoutReference }	{ settlement }
GET	/admin/audit-logs	Admin	—	{ data[] }

4. Order State Machine & Inventory Locking

4.1 Transition Table

From	To	Trigger	Guard
<i>(created)</i>	PENDING_PAYMENT	Buyer checks out, method = UPI/CARD/WALLET	inventory reserved in same TX
<i>(created)</i>	CONFIRMED	Buyer checks out, method = COD	inventory reserved in same TX
PENDING_PAYMENT	CONFIRMED	Razorpay webhook payment.captured	HMAC valid + idempotent insert
PENDING_PAYMENT	CANCELLED	Payment failed, or 15-min timeout cron	inventory released
CONFIRMED	DISPATCHED	Admin assigns an available agent	agentProfile.isAvailable = true
CONFIRMED	CANCELLED	Buyer or Admin cancels pre-dispatch	inventory released, refund initiated if paid online
DISPATCHED	OUT_FOR_DELIVERY	Agent taps "Start Delivery" (/pickup)	agent has active location permission
OUT_FOR_DELIVERY	DELIVERED	Agent taps "Mark Delivered" (/deliver)	ST_Distance(agent, shop) <= 200m
any non-terminal	CANCELLED	Admin override	reason required, written to AuditLog

DELIVERED and CANCELLED are terminal. Any transition not in this table throws 422 Unprocessable Entity — never silently allowed.

```
const ALLOWED: Record<OrderStatus, OrderStatus[]> = {
  PENDING_PAYMENT: ['CONFIRMED', 'CANCELLED'],
  CONFIRMED:      ['DISPATCHED', 'CANCELLED'],
  DISPATCHED:     ['OUT_FOR_DELIVERY', 'CANCELLED'],
  OUT_FOR_DELIVERY: ['DELIVERED'],
  DELIVERED:      [],
  CANCELLED:      [],
};
```

```
function assertTransition(current: OrderStatus, next: OrderStatus) {
```

```

if (!ALLOWED[current].includes(next)) {
  throw new HttpError(422, `Cannot move order from ${current} to ${next}`);
}
}

```

4.2 Inventory Locking (prevents overselling)

Pessimistic row lock via SELECT ... FOR UPDATE, inside one transaction that also creates the order — this is the part Prisma's query builder can't express, so it drops to raw SQL for the lock itself:

```

await prisma.$transaction(async (tx) => {
  const [inv] = await tx.$queryRaw<InventoryRow[]>`
    SELECT "quantityAvailable", "quantityReserved"
    FROM "ProductInventory"
    WHERE "productId" = ${productId}
    FOR UPDATE
  `;

  const remaining = inv.quantityAvailable - inv.quantityReserved;
  if (remaining < requestedQty) {
    throw new HttpError(409, 'Insufficient stock');
  }

  await tx.productInventory.update({
    where: { productId },
    data: { quantityReserved: { increment: requestedQty } },
  });

  const order = await tx.order.create({ data: { /* ... */ } });
  return order;
});

```

Mandatory test before Week 4 sign-off: 10 concurrent POST /orders requests for a product with exactly 1 unit in stock → exactly 1 succeeds (201), 9 return 409. The row lock, not application-level if checks, is what guarantees this.

4.3 Idempotent Payment Webhook

```
router.post('/payments/razorpay/webhook', express.raw({ type: '*' }), async (req, res) => {  
  const signature = req.headers['x-razorpay-signature'];  
  
  const expected = crypto.createHmac('sha256',  
    WEBHOOK_SECRET).update(req.body).digest('hex');  
  
  if (signature !== expected) return res.status(400).end();  
  
  try {  
    await prisma.payment.update({  
      where: { razorpayOrderId: payload.order_id },  
  
      data: { razorpayPaymentId: payload.payment_id, status: 'CAPTURED', capturedAt: new  
Date() },  
  
    });  
  
    // ... transition order PENDING_PAYMENT → CONFIRMED  
  } catch (e) {  
    if (e.code === 'P2002') return res.status(200).end(); // duplicate webhook, already  
processed  
  
    throw e;  
  }  
  
  res.status(200).end();  
});
```

`express.raw()` must be registered on this route **before** the global `express.json()` middleware, or HMAC verification will fail against a re-serialized body — a common, silent bug.

4.4 Geofence Check (delivery confirmation)

```
SELECT ST_Distance(  
  geography(ST_MakePoint(:agentLng, :agentLat)),  
  (SELECT location FROM "ShopProfile" sp  
    JOIN "Order" o ON o."buyerId" = sp."userId"  
    WHERE o.id = :orderId)  
) AS distance_meters;
```

`distance_meters <= 200` → allow DELIVERED; otherwise 422 with a clear "too far from shop" message the app can surface directly.

4.5 Restart-Safety (node-cron + JobRecovery)

Because background jobs run in-memory with no queue, anything that must survive a Railway restart is written to JobRecovery **before** the action that depends on it, and replayed on boot:

```
async function recoverOnStartup() {  
  const pending = await prisma.jobRecovery.findMany({ where: { status: 'PENDING' } });  
  for (const job of pending) {  
    await processJob(job); // e.g. re-check Razorpay order status via API  
  }  
}
```

5. Project Folder Structure

storefiller/

```
├── apps/  
│   ├── api/                # Express modular monolith  
│   │   ├── src/  
│   │   │   ├── modules/  
│   │   │   │   ├── auth/      # BE-1  
│   │   │   │   │   ├── auth.controller.ts  
│   │   │   │   │   ├── auth.service.ts  
│   │   │   │   │   └── auth.routes.ts  
│   │   │   │   ├── shop/      # BE-1  
│   │   │   │   ├── catalog/   # BE-2 (products, categories, search)  
│   │   │   │   ├── cart/      # BE-2  
│   │   │   │   ├── orders/    # BE-3 (FSM lives here)  
│   │   │   │   ├── payments/  # BE-3  
│   │   │   │   ├── wallet/    # BE-3  
│   │   │   │   ├── delivery/  # BE-2 (tracking + geofence)  
│   │   │   │   ├── support/   # BE-1  
│   │   │   │   ├── referral/  # BE-1  
│   │   │   │   └── notifications/ # BE-1 (FCM + device tokens)
```



```

| | | | └─ admin/          # BE-1 (dashboard, settlements, audit)
| | | └─ jobs/             # node-cron definitions
| | | | └─ payment-reconciliation.job.ts
| | | | └─ location-log-cleanup.job.ts
| | | | └─ startup-recovery.ts
| | | └─ middleware/
| | | | └─ auth.middleware.ts
| | | | └─ role-guard.middleware.ts
| | | | └─ error-handler.middleware.ts
| | | └─ lib/              # shared clients
| | | | └─ prisma.ts
| | | | └─ firebase-admin.ts
| | | | └─ cloudinary.ts
| | | | └─ razorpay.ts
| | | └─ utils/
| | | └─ server.ts
| | └─ prisma/
| | | └─ schema.prisma
| | | └─ migrations/
| | └─ package.json
| |
| └─ mobile/              # Expo React Native — MOB-1
|   └─ app/               # expo-router, file-based
|     └─ (auth)/          # phone entry, OTP, onboarding
|     └─ (buyer)/
|       └─ home.tsx
|       └─ category/[id].tsx
|       └─ product/[id].tsx
|       └─ cart.tsx
|       └─ checkout.tsx

```

```

├── orders/index.tsx
├── orders/[id]/track.tsx
├── profile.tsx
├── support.tsx
├── referral.tsx
├── (agent)/
├── deliveries.tsx
├── active-delivery/[id].tsx
├── (admin)/
├── catalog.tsx
├── orders.tsx
├── settlements.tsx
├── components/           # shared UI (ProductCard, OrderStatusStepper, etc.)
├── services/             # api client (axios + interceptors)
├── store/                # zustand slices
├── hooks/
├── constants/
├── assets/
├──
├── packages/
├── types/                # shared TS interfaces + zod schemas (api ↔ mobile)
├── config/               # shared eslint/tsconfig base

```

6. Tech Stack Stress-Test & Risk Matrix

You asked me to actually pressure-test this stack rather than just accept it — here's where it holds up, and where it needs a specific mitigation.

Component	Risk	Probability	Impact	Mitigation
Firebase Phone Auth	Production SMS delivery typically requires a Blaze (pay-as-you-go) billing account linked, even though the free monthly	Medium	Medium	Keep the email-OTP fallback always available; set a Firebase budget alert at ₹1 so any overage is

Component	Risk	Probability	Impact	Mitigation
	verification quota usually keeps actual spend at ₹0			caught immediately, not discovered on a bill
node-cron (in-memory)	If Railway ever runs >1 replica, every cron job fires once per replica — duplicate charges, duplicate notifications	Low (monolith, single service)	High	Pin replicas: 1 explicitly in Railway config; rely on JobRecovery (§4.5), not multiple instances, for resilience
Neon serverless compute-hours	Free tier has a finite monthly compute-hour budget. A connection pool held open 24/7 prevents autosuspend and can burn the budget in days, not weeks	Medium	High	Use Neon's pooled connection string, keep Prisma's pool idle timeout short, and monitor the Neon console weekly — B2B traffic is bursty (not always-on), so autosuspend between requests should hold
Railway vs. Render	Render's free tier sleeps after ~15 min idle, causing a 30–60s cold start on the next request — unacceptable for OTP login or 15s delivery polling	High (if Render chosen)	High	Recommend Railway as primary (usage-based \$5 free credit, no sleep); Render only for a staging/dev environment, never production
Cloudinary bandwidth	25GB/month free bandwidth could be eaten by unoptimized product images at scale	Low at MVP	Low	Apply <code>f_auto,q_auto</code> transformation params on every delivery URL; monitor via Cloudinary dashboard
Razorpay live mode	Live (non-test) mode requires a registered business PAN/GST — individual accounts are capped to test mode	Medium	Medium	COD is already a first-class payment method, so orders aren't blocked while business KYC/activation completes
PDFKit, no object storage	None — receipts are generated in-memory per request from Order/OrderItem data already in Postgres, so	—	—	This is a genuine improvement over storing receipts in Supabase/S3: nothing to clean up, nothing to expire

Component	Risk	Probability	Impact	Mitigation
	there's no storage growth risk at all			
15s polling	Fine at the concurrency B2B delivery implies (tens of concurrent active deliveries, not thousands)	Low	Low	If concurrent active deliveries ever exceed ~300–500, revisit with Server-Sent Events before adding a paid WebSocket layer
Single Neon branch, no replica	A schema migration mistake affects production directly	Medium	Medium	Use a Neon branch (free, instant) as a staging copy for every migration before applying to main

7. Free-Tier Quota Quick Reference

Service	Free Limit	Expected MVP Usage	Alert At	Fallback
Neon PostgreSQL	~0.5GB storage, limited compute-hrs/mo	Low (single warehouse catalog)	400MB / 80% compute budget	Archive delivered orders >90 days; upgrade if needed
Railway	\$5 credit/month	~\$3–4/mo at 1 service	Deploy failure	Optimize image size, prune deps
Cloudinary	25GB bandwidth/mo	~5GB (product + shop photos)	20GB	f_auto,q_auto on all URLs
Firebase Phone Auth	Free tier verification quota	Low (store-owner volume, not consumer)	Any unexpected charge	Email OTP fallback
Razorpay	No monthly fee, ~2% per txn	Pay-per-order	—	COD always available
Resend (email OTP)	3,000 emails/mo	<100/mo at MVP 2,500/mo		Batch/queue non-critical emails

Closing Note

This is a Markdown reference — every code block here is meant to be copy-pasted straight into your repo. If you'd like this repackaged as a branded PDF or Word doc (in the style of your Kisaan Connect roadmap) for stakeholder circulation, I can generate that as a follow-up.

